



Conception et mise en place d'un système de détection et de réponse aux attaques ransomware dans un environnement contrôlé

Codjo Fréjus Loïc SANGRONIO

Licence Informatique — Option Sécurité Informatique

24 juin 2026

Encadreur :

Ing. Guy KPEDJO

Maître de stage :

[Nom du maître de stage]



1. Introduction Générale
2. Revue de littérature
3. Analyse et Conception
4. Réalisation et Résultats
5. Conclusion et Perspectives

Présentation : **10 min**

Diapositives : **21**

Démo live : **5 min**

VMs test : 3 KVM





01

🎯 Introduction Générale



Introduction Générale

Contexte

L'évolution rapide du numérique expose les organisations aux ransomwares : des logiciels malveillants qui rendent les données inaccessibles puis exigent une rançon.

- **+73 %** d'attaques signalées en 2023 (*ENISA*)
- Délai moyen de détection **sans outil dédié : 21 jours**
- Coût moyen par attaque : **> 1 million de dollars**
- Cibles : PME, hôpitaux, administrations, universités

Problématique

Les terminaux constituent des points d'entrée sensibles, tandis que les antivirus basés sur les signatures restent insuffisants face aux variantes nouvelles ou inconnues. La nécessité de détecter automatiquement un comportement ransomware sur un parc de machines hétérogène — et de déclencher une réponse maîtrisée **sans déployer de vrai malware** — constitue le défi central de ce travail.

Introduction Générale

Objectif général

Concevoir et mettre en œuvre un **système de détection et de réponse aux attaques ransomware** basé sur la surveillance comportementale des fichiers et des processus, avec une console de supervision traçable.

Objectifs spécifiques

- Surveiller les événements fichiers en temps réel sur les terminaux
- Analyser les comportements et calculer un score de risque configurable
- Générer automatiquement des alertes et des incidents de sécurité
- Proposer et exécuter des actions de protection avec **contrôle humain**
- Sécuriser la console par une authentification forte (2FA TOTP RFC 6238)
- Valider le pipeline à l'aide de scénarios d'attaque représentatifs

Organisation du mémoire

Chapitre I Revue de littérature

Sécurité informatique, logiciels malveillants, ransomwares, antivirus, IDS, EDR : positionnement de RansomShield.

Chapitre II Analyse et Conception

Besoins fonctionnels et non fonctionnels, architecture générale, modèle de données, processus de détection et mesures de sécurité.

Chapitre III Réalisation et Résultats

Implémentation des composants, scénarios de validation, résultats obtenus, limites et perspectives.

Mémoire rédigé en $\text{\LaTeX}$ selon le canevas IFRI — Licence Informatique, option Sécurité Informatique — Année 2023–2024





02



Revue de littérature



Définition d'un ransomware

Un ransomware est un logiciel malveillant qui vise à bloquer l'accès à un système ou à des données, souvent par chiffrement, puis à réclamer une rançon à la victime. (*NIST SP 1800-25, CISA*)

Comportements caractéristiques — détectables avant les dommages irréversibles

- Renommage massif de fichiers avec extension suspecte (.locked, .enc, .crypt...)
- Création d'une note de rançon (HOW_TO_DECRYPT.txt)
- Suppression des clichés instantanés VSS (`vssadmin delete shadows /all`)
- Utilisation d'outils légitimes détournés — LOLBins (`bcdedit`, `wmic`, `vssadmin`)
- Suppression massive de fichiers, accès réseau inhabituel

Ces signaux constituent la base de la **détection comportementale**.

Quelques solutions existantes

- **Antivirus** — détection par signatures statiques (ClamAV, Windows Defender)
- **SIEM génériques** — collecte de journaux sans corrélation comportementale
- **EDR commerciaux** — CrowdStrike Falcon, SentinelOne, Microsoft Defender XDR

Limites des solutions existantes

- ❌ Contournables par variantes nouvelles, obfuscation ou LOLBins
- ❌ Coût élevé des EDR commerciaux (hors portée académique)
- ❌ Absence d'outil open-source académique démontrable en conditions réelles

Notre réponse : RansomShield

Approche comportementale open-source, multi-plateforme, démontrable sur machines virtuelles réelles — **contrôle humain à chaque étape.**



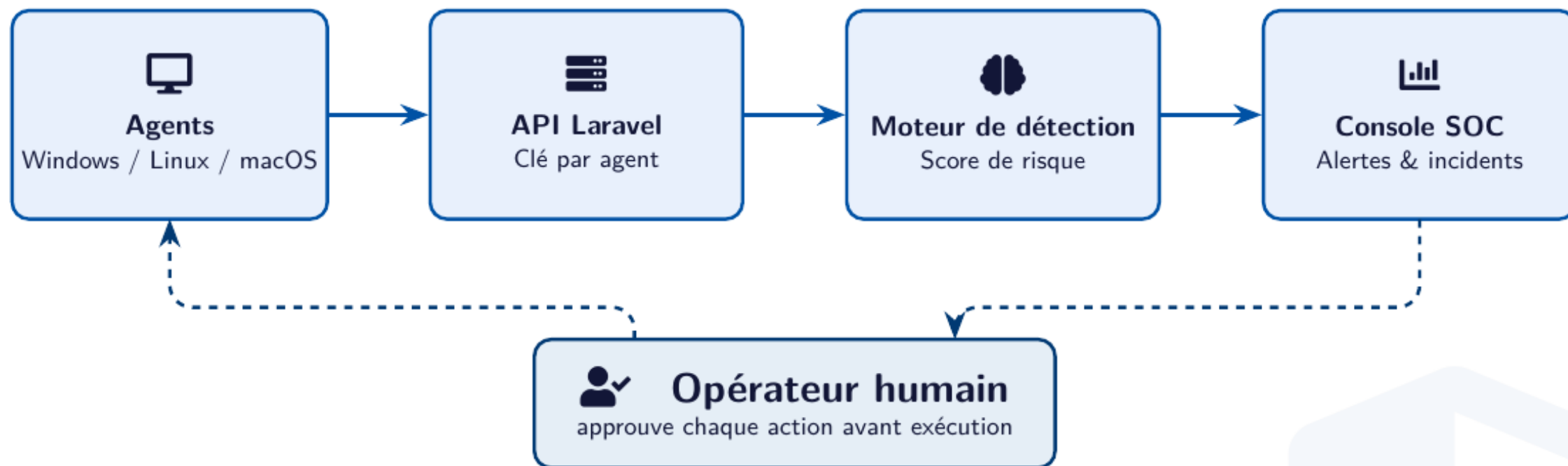
03



Analyse et Conception



Architecture générale de RansomShield



Aucune action n'est exécutée automatiquement — l'opérateur valide toujours avant.



Les quatre composants du système

Agent Python

Surveille les dossiers via **Watchdog** / **inotify**. File locale SQLite (résilience réseau). Enrôlement par token à usage unique. Service systemd — démarrage automatique.

Découverte réseau

fping (ICMP) + ARP sweep sur les /24. Scan automatique toutes les 5 minutes. Aucun enrôlement sans **validation manuelle admin**.

Console SOC Laravel

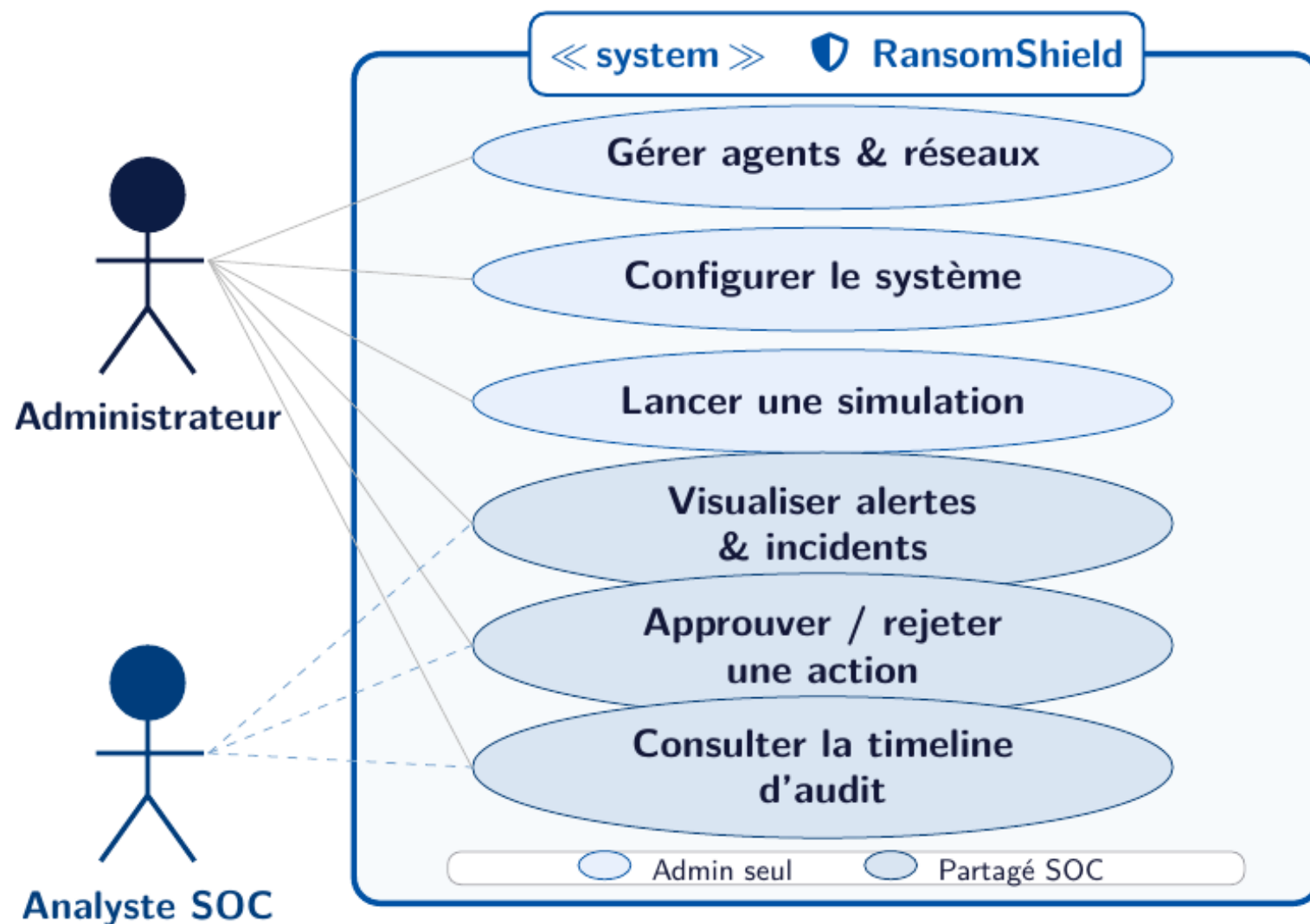
Dashboard temps réel (Chart.js, AJAX 30 s). Alertes, incidents, timeline, file d'approbation. Notifications : web, e-mail, son, webhook.

Module de simulation

5 scénarios d'attaque, jusqu'à **22 événements**. Tous marqués `is_simulation=true` — nettoyage facile après chaque démo.



Diagramme de cas d'utilisation



Modèle de données

Tables principales

agents	enrôlement, état, risque
events	événements fichiers
alerts	score, niveau, signaux
incidents	fiche, statut, timeline
detection_rules	règles configurables
protection_actions	décisions SOC
audit_logs	traçabilité horodatée
simulation_runs	profils d'attaque

Relations clés

- `agents` → `events` (1-N)
- `events` → `alerts` (N-1)
- `alerts` → `incidents` (N-1)
- `incidents` → `protection_actions` (1-N)
- `incidents` → `audit_logs` (1-N)
- `simulation_runs` → `events` (1-N)

8 tables MySQL — pipeline complet : événement → alerte → incident → action → audit



04



Réalisation et Résultats



Choix techniques

Backend & Base de données

- 📄 **Laravel 11** / PHP 8.3
- 🗄️ **MySQL 8.0** (Eloquent ORM)
- 🔗 **API REST** sécurisée (clé per-agent)

Agent de surveillance

- 🔗 **Python 3.10**
- 🔍 **Watchdog** (inotify Linux)
- 🗄️ **SQLite** (file locale offline)
- 📁 **psutil** (processus)

Frontend

- 🖥️ **Blade** (templates Laravel)
- 📊 **Chart.js** (intégré local)
- 🔄 **AJAX** toutes les 30 s

Infrastructure de test

- 📄 **3 VMs KVM/libvirt**
- 🌐 **Réseau 10.20.0.0/24**
- 🔒 **2FA TOTP RFC 6238**
- 🔑 **Token usage unique (enrôlement)**

Agent Python et moteur de détection

Cycle de vie de l'agent

- 1 Démarrage : lecture .env
- 2 Enrôlement : token → clé API
- 3 Watchdog observe les dossiers
- 4 Événement → file SQLite locale
- 5 POST /api/agent/events
- 6 Heartbeat toutes les 30 s

Installation en une commande (Linux, Windows, macOS) :

```
curl -sSL http://<soc>/install.sh | bash
```

Scoring comportemental

Signal détecté	Score
Extension suspecte (.locked)	+80
Note de rançon créée	+55
Suppression clichés VSS	+60
LOLBin suspect	+45
Suppression massive fichiers	+50

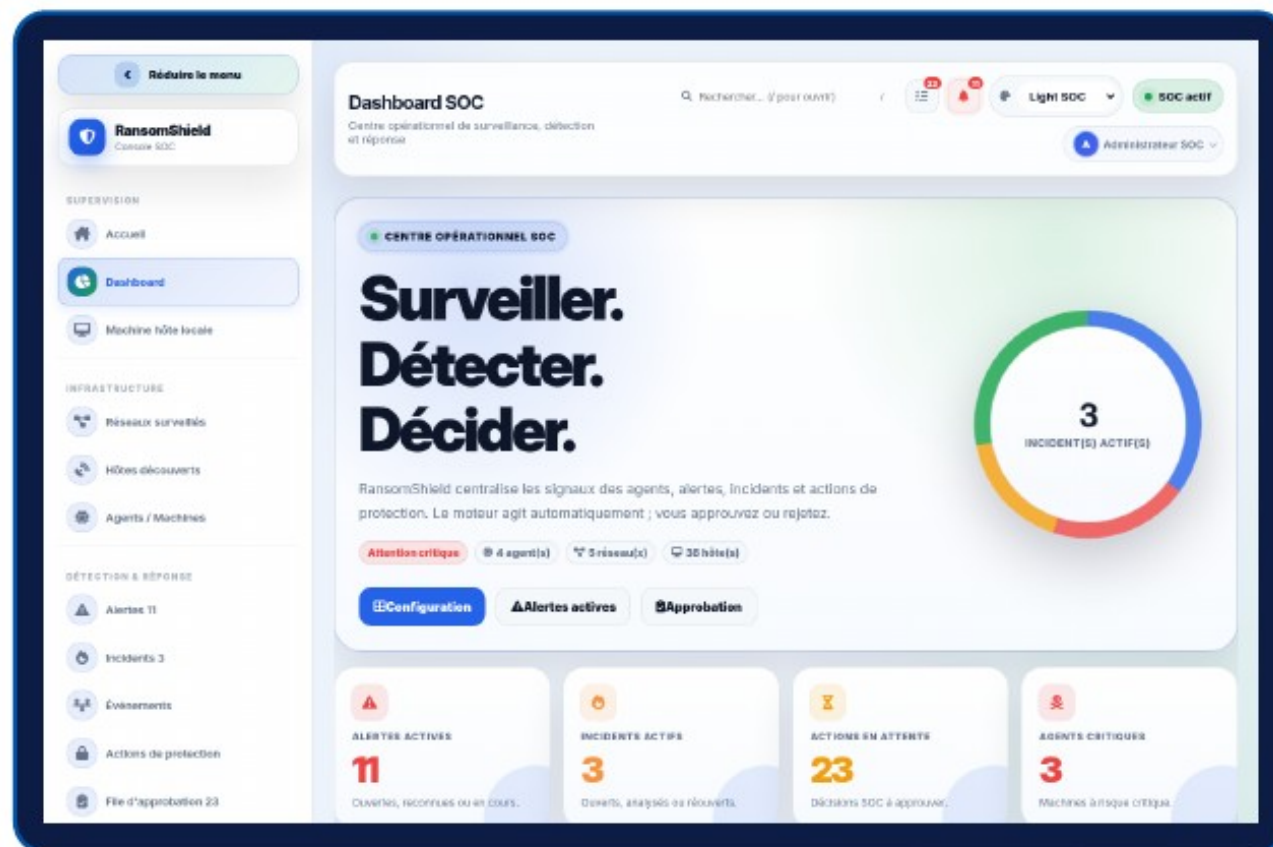
Score < 30	Normal
30 à 59	Suspect
60 à 99	Élevé
≥ 100	Critique

Alerte + incident créés automatiquement en < 5 s

Console SOC — Interface de supervision

Pages de la console

- 🏠 Tableau de bord
- 👤 Agents (état, heartbeat)
- 🔔 Alertes (score, signaux)
- 📁 Incidents
- 🕒 Timeline d'audit
- ✅ File d'approbation
- 🔧 Règles (CRUD live)
- 🔍 Recherche globale
- 🔔 Notifications
- ⚙️ Paramètres système



🔄 Mise à jour AJAX toutes les 30 s · 🔔 Web · E-mail SMTP · Son · Webhook

Tests de validation et résultats

5 scénarios d'attaque testés

Scénario	Évts	Résultat
Chiffrement basique	7	✓
Kill chain complète	22	✓
Chiffrement massif	18	✓
Exfiltration données	8	✓
LOLBins suspects	5	✓

Environnement : 3

VMs KVM — réseau 10.20.0.0/24

Résultats obtenus

- ✓ Détection en < 5 secondes (tous scénarios)
- ✓ Aucun événement perdu (file SQLite)
- ✓ Pipeline complet validé : événement → alerte → incident → timeline
- ✓ Rollback isolation réseau fonctionnel
- ✓ 2FA TOTP et API sécurisée (per-agent)

Limite identifiée

Tests conduits sur Linux uniquement. Un attaquant ralentissant son rythme réduit la visibilité.



Démonstration Live

5 minutes — système réel sur VMs KVM



05



Conclusion et Perspectives



Conclusion et Perspectives

Conclusion

RansomShield est une solution opérationnelle couvrant l'ensemble du cycle de traitement d'un incident : **collecte** → **analyse** → **alerte** → **réponse** → **audit**.

- ✓ Agent Python multi-plateforme — installation en une commande
- ✓ Console SOC : simulation, 4 canaux de notification, rapports PDF, 2FA
- ✓ Moteur de détection : scoring configurable, règles CRUD, < 5 s
- ✓ Environnement contrôlé : aucun vrai malware déployé

Perspectives d'évolution

- **HTTPS** — chiffrement TLS en production
- **RBAC** — rôles distincts (administrateur, analyste SOC)
- **Machine Learning** — Isolation Forest (baseline comportementale)
- **MITRE ATT&CK** — alignement et couverture du référentiel
- **Intégration SIEM** — export Syslog vers Splunk / ELK Stack

Merci de votre aimable attention !



Codjo Fréjus Loïc SANGRONIO

lsangronio@gmail.com | IFRI-UAC 2023–2024

Encadreur : Ing. Guy KPEDJO

Annexe A — File d'approbation et rollback

Principe

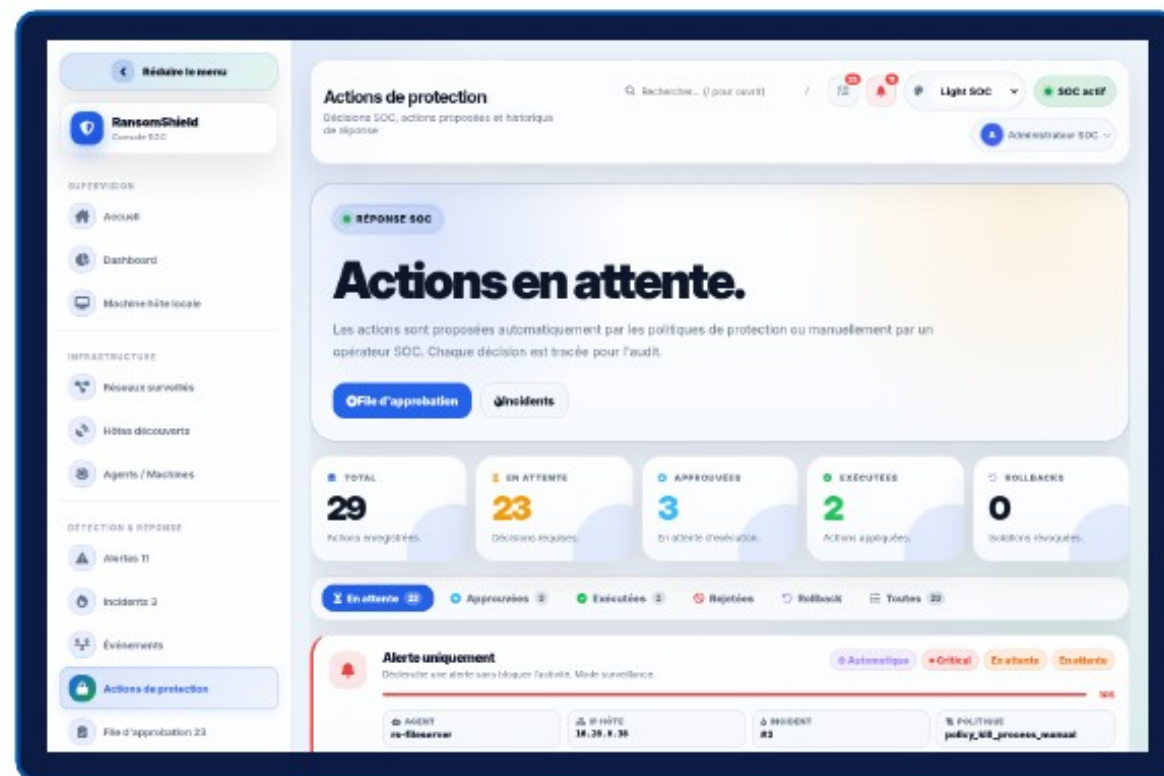
Le moteur **propose** — l'opérateur **décide**.

Chaque action affiche :

- Type (scan, isolation, quarantaine)
- Niveau de risque de l'incident
- Agent concerné

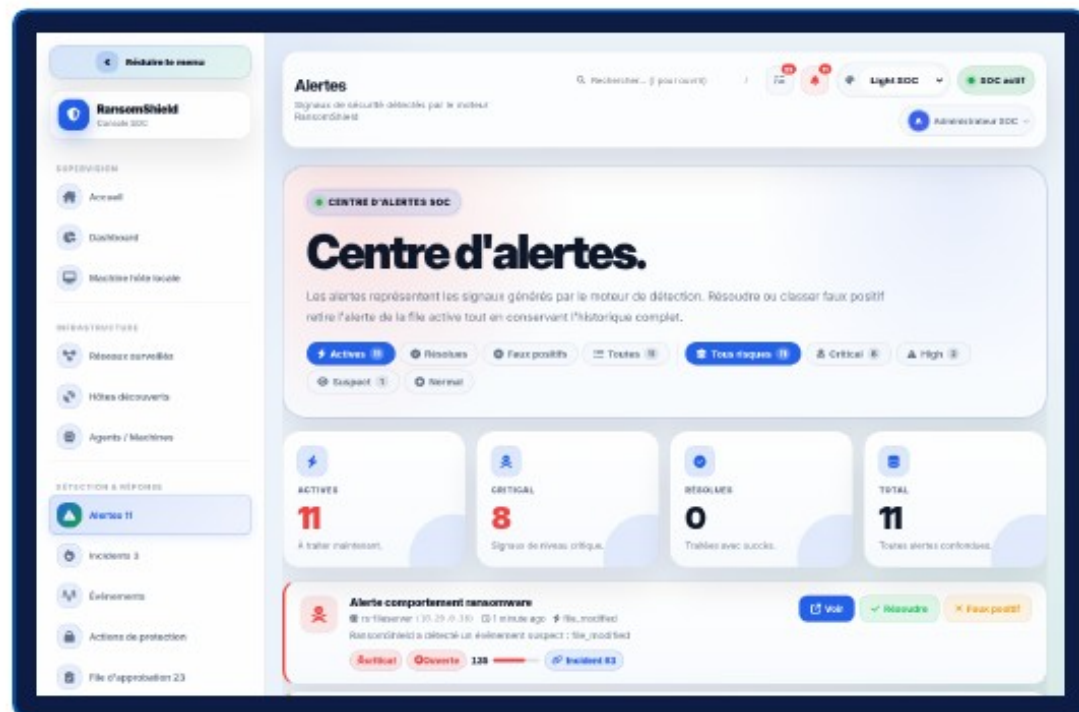
Boutons **Approuver** / **Rejeter** — tout est horodaté.

Rollback possible sur l'isolation réseau.

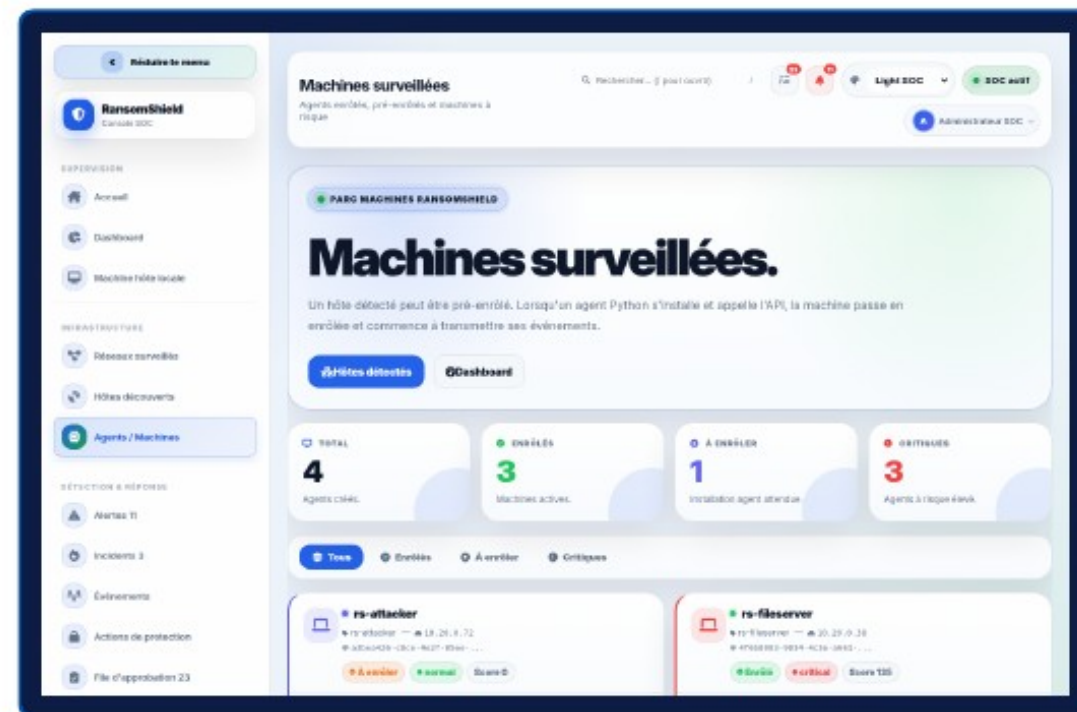


Annexe B — Captures d'écran

Alertes



Agents



Timeline incident



Simulation en cours

